

SIGNAL PROCESSING / DEEP LEARNING / PYTORCH

Wavelet-SRNet in PyTorch

Haar wavelet packet reconstruction with an identity loss

Gerardo Lauro · Antonio Maria Improta · Francesco Simone Pirozzi

Multimedia Signal Processing project | Università di Napoli Federico II

Course instructor: Prof. Luisa Verdoliva

1 Objective and contribution

This project studies and reimplements Wavelet-SRNet [1], a CNN that predicts wavelet coefficients before reconstructing a high-resolution face. The configured task is $8\times$ face super-resolution: a 16×16 RGB input is mapped to a 128×128 image. The work combines a fixed signal transform with a learned estimator and a training objective spanning wavelet, image and identity representations.

The project contribution comprises three parts:

- **Paper-to-code implementation.** A modular PyTorch model with explicit coefficient ordering, generated Haar filters and a differentiable, fixed inverse transform. The wavelet reconstruction principle is inherited from the original method.
- **Identity objective.** A cosine-distance loss on frozen ArcFace R50 embeddings extends the wavelet and image losses. A separate R100 backbone evaluates identity.
- **Experimental engineering.** Validated YAML configurations, decoded-image caching, mixed-precision training, checkpoint recovery, profiling and matched evaluation tools.

2 Signal representation and original Wavelet-SRNet formulation

Wavelet packet transform

Following Huang *et al.* [1], the method is based on the **wavelet packet transform (WPT)**, which decomposes an image into a sequence of wavelet coefficients of the same spatial size. The original paper uses the Haar wavelet and computes the coefficients with a 2-D fast wavelet transform. At each level, both approximation and detail coefficients are decomposed again, producing a full wavelet packet representation. In the paper’s interpretation, approximation coefficients encode global face topology, while detail coefficients describe structure and texture.

For an n -level full wavelet packet decomposition, the magnification factor r and number of wavelet coefficients N_w are

$$r = 2^n, \quad N_w = 4^n. \quad (1)$$

For the configured $8\times$ case, $n = 3$ and $N_w = 64$. Each coefficient image has the same spatial dimensions as the LR input and contains three RGB channels; the implementation therefore stores $64 \times 3 = 192$ coefficient channels.

Wavelet-based super-resolution

The original paper first formulates generic single-image super-resolution as learning a mapping $f_\theta(x)$ from a low-resolution input x to a high-resolution estimate $\hat{y}$. Given a dataset $D \equiv \{(x_i, y_i)\}_{i=1}^N$, the parameter optimization is written as

$$\arg \max_{\theta} \sum_{(x,y) \in D} \log p(y | x). \quad (2)$$

The conventional pixel-space objective is the mean-squared error

$$l_{\text{mse}}(\hat{y}, y) = \|\hat{y} - y\|_F^2. \quad (3)$$

Huang *et al.* argue that minimizing only image-space MSE tends to miss high-frequency texture details. They therefore transform the SR problem from image space to the wavelet domain. Let

$$C = (c_1, c_2, \dots, c_{N_w}), \quad \hat{C} = (\hat{c}_1, \hat{c}_2, \dots, \hat{c}_{N_w})$$

denote the ground-truth and inferred wavelet coefficients. The learned mapping g_θ predicts the coefficient set, and the corresponding optimization is

$$\arg \max_{\theta} \sum_{(x,C) \in D} \log p(C | x). \quad (4)$$

This is the central formulation inherited by the project: the CNN predicts wavelet coefficients from the LR image, then a fixed inverse Haar transform reconstructs the SR image.

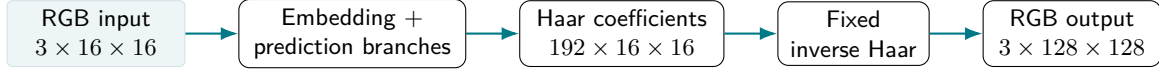


Figure 1. Implemented $8\times$ inference path; batch dimension omitted.

Component	Current configuration
Embedding	3×3 stem; residual stages with 64, 128, 256, 512 and 1024 channels, two residual blocks per stage; batch normalization and ReLU.
Prediction	Four branches producing 1, 3, 12 and 48 bands; 32 initial hidden channels per band and one residual block after each interim block.
Representation	64 RGB bands form 192 channels, ordered by coefficient (R, G, B for each band). Feature maps retain the LR spatial dimensions.

Implementation: `models/waveletsrnet.py`; `configs/config_8x.yaml`.

3 Training objective

Original wavelet-domain losses

The original Wavelet-SRNet paper defines two losses in the wavelet domain. The **wavelet prediction loss** is a weighted MSE:

$$l_{\text{wavelet}}(C, \hat{C}) = \left\| W^{1/2} \odot (\hat{C} - C) \right\|_F^2 = \sum_{i=1}^{N_w} \lambda_i \|\hat{c}_i - c_i\|_F^2. \quad (5)$$

Here $W = (\lambda_1, \dots, \lambda_{N_w})$ balances the importance of different wavelet bands. Larger weights can emphasize high-frequency coefficients, while the approximation coefficient constrains global topology.

The paper’s **texture loss** is designed to prevent high-frequency coefficients from collapsing toward zero:

$$l_{\text{texture}} = \sum_{i=k}^{N_w} \gamma_i \max(\alpha \|c_i\|_F^2 + \varepsilon - \|\hat{c}_i\|_F^2, 0). \quad (6)$$

A full-image MSE is added after reconstruction. With R denoting the reconstruction matrix and $\hat{y} = R\hat{C}$, the original unified loss is

$$l_{\text{total}} = l_{\text{wavelet}} + \mu l_{\text{texture}} + \nu l_{\text{full-image}} = \sum_{i=1}^{N_w} \lambda_i \|\hat{c}_i - c_i\|_F^2 + \mu \sum_{i=k}^{N_w} \gamma_i \max(\alpha \|c_i\|_F^2 + \varepsilon - \|\hat{c}_i\|_F^2, 0) + \nu \|R\hat{C} - y\|_F^2. \quad (7)$$

These equations reproduce the formulation used in the original paper; the PyTorch project preserves the same roles but implements them with tensor-wise reductions and adds the identity term described below.

Current implementation and identity loss

The current configuration combines four reconstruction terms with a task-specific identity term:

$$L = 0.01L_{\text{low}} + L_{\text{high}} + L_{\text{tex}} + 0.1L_{\text{img}} + 0.03L_{\text{ID}}. \quad (8)$$

The low-frequency MSE uses the first RGB band; the high-frequency MSE uses the remaining 63 bands. The image term $L_{\text{img}} = \text{MSE}(\hat{y}, y)$ is evaluated before output clipping.

For batch item b , high-frequency band k and spatial position p , define local RGB energy

$$E_{bkp} = \sum_{c \in \{R, G, B\}} C_{bkcp}^2, \quad \hat{E}_{bkp} = \sum_{c \in \{R, G, B\}} \hat{C}_{bkcp}^2.$$

The implemented texture penalty is

$$L_{\text{tex}} = \text{mean}_{b,k,p} \left[\max(0, 1.2E_{bkp} - \hat{E}_{bkp}) \right]. \quad (9)$$

Let $z(v) = f_{R50}(v) / \|f_{R50}(v)\|_2$ be the normalized embedding of the frozen ArcFace-compatible R50 backbone [2]. The identity loss is

$$L_{\text{ID}} = \frac{1}{B} \sum_{b=1}^B [1 - z(\hat{y}_b)^\top z(y_b)]. \quad (10)$$

R50 remains frozen in evaluation mode. Gradients flow through the SR embedding to the trainable CNN, while the HR embedding is computed without gradients. A separate frozen R100 backbone is used for identity evaluation.

Implementation: `losses/__init__.py`; `models/arcface.py`; `train.py`.

4 Pipeline efficiency and maintainability

Implemented mechanism	Engineering purpose and scope
Decoded-image cache	A file-backed uint8 memory map stores resized images. Workers reuse decoded data across epochs, avoiding repeated JPEG decoding and resizing.
Data loading	Three workers, persistent workers, prefetch factor 2 and CUDA pinned-memory transfers are configured; each worker opens the cache lazily.
Mixed precision	CUDA AMP covers the CNN forward pass and Haar synthesis. Wavelet targets and reconstruction-loss evaluation use FP32; a gradient scaler handles updates.
Profiling	Warmup and CUDA-synchronized stage timings separate data wait, transfers, forward pass, loss, backward pass and optimizer work. A synthetic-data mode helps isolate compute from input loading.
Configuration and recovery	YAML inheritance and validation centralize architecture settings. Checkpoints include model, optimizer, scheduler, scaler, configuration and RNG states; CSV logs record loss components and validation metrics.

Measurement boundary. `tools/benchmark.py` measures the wavelet training path, excluding ArcFace identity loss, so its timings do not cover the full extended training loop. TF32 is enabled where supported; channels-last, cuDNN autotuning and `torch.compile` are disabled in the main configuration.

Implementation: `dataset.py`; `utils/`; `tools/benchmark.py`.

5 Experimental protocol

Data and preprocessing

The main configuration targets aligned CelebA images. Sorted image paths are shuffled with seed 42 and split by image into approximately 80% training, 10% validation and 10% test. This is a project-specific random split rather than the official partition; identities are not used to enforce disjoint splits.

HR preparation resizes the shorter side bicubically and takes a central square crop at 128×128 . Tensors are RGB in $[0, 1]$. LR targets are generated with antialiased bicubic downsampling to 16×16 , followed by clipping to $[0, 1]$. Training uses horizontal flips with probability 0.5; random cropping is disabled. Validation and test use neither augmentation.

For cross-dataset evaluation, `tools/prep_helen.py` prepares HELEN face crops from parsing masks: facial labels 1–9 determine a bounding box, expanded by a factor of 1.3 to a square and resized to 128×128 . This is mask-based cropping, not landmark alignment.

Training configuration and controlled comparison

Setting	Configured value
Optimization	Adam; base LR 2×10^{-4} ; $\beta = (0.9, 0.999)$; weight decay 5×10^{-4} .
Schedule	Up to 70 epochs; batch size 128; cosine schedule with 2 warmup epochs and minimum LR 10^{-6} .
Validation and selection	Every epoch, first 20 validation batches (up to 2,560 images); best checkpoint by validation PSNR; early-stopping patience 20.
Baseline (E0)	Bicubic upsampling of the same LR images; no training.
Wavelet model (E1)	<code>config_8x_noid.yaml</code> : inherited settings with identity weight 0.
Identity extension (E2)	<code>config_8x.yaml</code> : identity weight 0.03; frozen R50 for the loss, separate frozen R100 for identity evaluation.

Table 1. Configuration values describe the supplied protocol, not a record of completed runs.

Evaluation definitions

Metric	Implementation in the full evaluation script
PSNR $\uparrow$	Per-image MSE on $Y = 0.299R + 0.587G + 0.114B$, with range 1; PSNR values are aggregated across images.
SSIM $\uparrow$	RGB; 11×11 Gaussian window with $\sigma = 1.5$ and zero padding; spatial and channel mean per image.
LPIPS $\downarrow$	AlexNet-based LPIPS [3] by default; RGB values mapped from $[0, 1]$ to $[-1, 1]$.
Identity $\uparrow$	Cosine similarity between normalized SR and HR embeddings from the separate ArcFace R100 evaluator.

Predictions are clipped to $[0, 1]$ for evaluation. `tools/eval_paper.py` reports per-image aggregate means and standard deviations for the model, bicubic, bilinear and nearest-neighbor baselines. `test.py` reports PSNR, SSIM and optional LPIPS, but does not evaluate ArcFace.

Implementation: `metrics/__init__.py`; `train.py`; `tools/eval_paper.py`; `tools/stat_test.py`.

6 Reproduction and evidence

Repository entry points

Path	Responsibility
<code>models/waveletsrnet.py</code>	Residual CNN, Haar analysis/synthesis and model factory.
<code>models/arcface.py</code>	Face-embedding backbones, weight loading and preprocessing.
<code>losses/__init__.py</code>	Wavelet, texture, image and identity losses.
<code>dataset.py</code> ; <code>metrics/__init__.py</code>	Data pipeline and reconstruction/perceptual metrics.
<code>train.py</code> ; <code>test.py</code> ; <code>utils/</code>	Training, basic inference, configuration, checkpoints and logging.
<code>tools/</code> ; <code>tests/</code>	Full evaluation, profiling, diagnostics and automated checks.

Minimal experiment workflow

```
python train.py --config configs/config_8x_noid.yaml
python train.py --config configs/config_8x.yaml
```

Evaluate each best checkpoint with `tools/eval_paper.py`, using its matching configuration and a distinct JSON output. Evaluation requires LPIPS and pretrained ArcFace weights. Confirm that the startup log reports the R50 loss as enabled before interpreting a run as E2.

Verification and available evidence

The supplied tests cover Haar reconstruction, low-band ordering, network/output shapes, data loading, configuration validation, checkpoint round trips and a small-data overfitting check. The inspected snapshot contains 64 comparison images, but no checkpoints, epoch logs or evaluation JSON files establishing their provenance or the quantitative E1/E2 comparison. This report therefore documents implementation and protocol; accuracy, identity gains and throughput remain unverified.

Technical value. The project integrates a structured signal representation with differentiable learning and an inspectable PyTorch pipeline. Transferable skills include filter-bank reasoning, paper reimplementations, loss design, data-path efficiency, numerical precision and profiling. The application remains face super-resolution.

References

- [1] H. Huang, R. He, Z. Sun and T. Tan. *Wavelet-SRNet: A Wavelet-Based CNN for Multi-Scale Face Super Resolution*. ICCV, pp. 1689–1697, 2017.
- [2] J. Deng, J. Guo, N. Xue and S. Zafeiriou. *ArcFace: Additive Angular Margin Loss for Deep Face Recognition*. CVPR, 2019.
- [3] R. Zhang, P. Isola, A. A. Efros, E. Shechtman and O. Wang. *The Unreasonable Effectiveness of Deep Features as a Perceptual Metric*. CVPR, pp. 586–595, 2018.